Question 1: Hospital Shift Scheduling (Backtracking Search)

1.1 Problem formulation as a CSP

Component	Definition
Variables	D1, D2, D3 (the shift of each doctor)
Domain	{Morning, Afternoon, Night}, ordered Morning < Afternoon < Night
C(i)	D1 ≠ Night
C(ii)	D2 works before D3, i.e. shift(D2) < shift(D3)
C(iii)	One doctor per shift (all-different)
C(iv)	D3 ≠ Morning
C(v)	Each doctor is assigned exactly one shift (complete assignment)

1.2 Backtracking algorithm

Backtracking is depth-first search over partial assignments. At each level one doctor is assigned a shift, every constraint that involves only assigned doctors is checked, and the branch is abandoned (backtracked) as soon as a check fails. Variable order: D1, D2, D3. Value order: Morning, Afternoon, Night.

```
BACKTRACK(assignment):
    if every doctor is assigned: return assignment
    D <- next unassigned doctor
    for shift in [Morning, Afternoon, Night]:
        if CONSISTENT(D = shift, assignment):   # checks C(i)..C(v)
            assignment[D] <- shift
            result <- BACKTRACK(assignment)
            if result != failure: return result
            remove D from assignment            # backtrack
    return failure
```

1.3 Step-by-step trace with constraint checks

Step	Try	Constraint checks	Outcome
1	D1 = Morning	C(i) D1 ≠ Night ✓; C(iii) Morning is free ✓	Accept
2	D2 = Morning	C(iii) Morning already used by D1 ✗	Reject
3	D2 = Afternoon	C(iii) Afternoon is free ✓; C(ii) leaves Night for D3, so D2 before D3 is still possible ✓	Accept
4	D3 = Morning	C(iii) taken by D1 ✗; C(iv) D3 ≠ Morning ✗	Reject
5	D3 = Afternoon	C(iii) taken by D2 ✗; C(ii) Afternoon is not before Afternoon ✗	Reject
6	D3 = Night	C(iv) ✓; C(ii) Afternoon < Night ✓; C(iii) ✓; C(v) all three assigned ✓	Accept: goal reached

Search tree (all branches)

```
root
|-- D1 = Morning      [OK]
|   |-- D2 = Morning      [X] shift already taken           C(iii)
|   |-- D2 = Afternoon    [OK]
|   |   |-- D3 = Morning      [X] taken / D3 cannot work Morning  C(iii), C(iv)
|   |   |-- D3 = Afternoon    [X] taken, and not after D2         C(iii), C(ii)
|   |   `-- D3 = Night        [OK]  ==> SOLUTION 1
|   `-- D2 = Night        [dead end] D3 must be after Night       C(ii)
|-- D1 = Afternoon    [OK]
|   |-- D2 = Morning      [OK]
|   |   `-- D3 = Night        [OK]  ==> SOLUTION 2
|   |-- D2 = Afternoon    [X] taken                          C(iii)
|   `-- D2 = Night        [dead end] D3 must be after Night       C(ii)
`-- D1 = Night        [X] D1 cannot work Night                 C(i)
```

1.4 Final answer

Valid schedule (first solution found by backtracking)

Morning: D1 | Afternoon: D2 | Night: D3

All constraints hold: D1 is not on Night ✓; D2 (Afternoon) is before D3 (Night) ✓; one doctor per shift ✓; D3 is not on Morning ✓; every doctor has exactly one shift ✓.

Why Night must go to D3: D1 is barred from Night by C(i), and D2 cannot take Night because D3 would then have to work after Night. That forces D3 onto Night in every solution. The only freedom is how D1 and D2 share Morning and Afternoon, so the problem has exactly two solutions. The second one is D2 = Morning, D1 = Afternoon, D3 = Night. If the paper asks for a single schedule, give the first one above.

Question 2: Grid Navigation with BFS / A* and the Manhattan Heuristic

2.1 Problem setup

The robot moves on a 5×5 grid from S = (0,0) to G = (4,4). Obstacles (assumed, since the figure was not provided): (1,1), (1,3), (2,1), (3,3), (4,2). Moves are Up, Down, Left, Right at cost 1 each, and obstacles cannot be entered.

	c0	c1	c2	c3	c4
r0	S				
r1		#		#	
r2		#			
r3				#	
r4			#		G

S = start, G = goal, # = obstacle

> **BFS versus the priority queue in the question**
>
> Plain BFS uses a FIFO queue and no heuristic. A priority queue ordered by a Manhattan-distance heuristic is A* search, with f(n) = g(n) + h(n). The solution below runs **A*** because the question asks for priority-queue updates and the Manhattan heuristic, and then runs **BFS** on the same grid for comparison.
>
> Heuristic: h(n) = |row − 4| + |col − 4|. Every move costs 1 and moves are 4-directional, so h never overestimates the true cost (admissible) and satisfies the triangle inequality (consistent). A* with such a heuristic returns an optimal path.

Conventions: the priority queue is ordered by smallest f, then smallest h, then first inserted. Neighbours are generated in the order Up, Down, Left, Right. A node already expanded is never re-expanded. If a cheaper route to a queued node is found, its entry is replaced.

2.2 A* step-by-step node expansion

Step	Expand	Neighbours added to the queue	Priority queue after the step (best first)
1	(0,0) g=0, h=8, f=8	(1,0) g=1, h=7, f=8 (0,1) g=1, h=7, f=8	(1,0)[f=8] , (0,1)[f=8]
2	(1,0) g=1, h=7, f=8	(2,0) g=2, h=6, f=8	(2,0)[f=8] , (0,1)[f=8]
3	(2,0) g=2, h=6, f=8	(3,0) g=3, h=5, f=8	(3,0)[f=8] , (0,1)[f=8]
4	(3,0) g=3, h=5, f=8	(4,0) g=4, h=4, f=8 (3,1) g=4, h=4, f=8	(4,0)[f=8] , (3,1)[f=8] , (0,1)[f=8]
5	(4,0) g=4, h=4, f=8	(4,1) g=5, h=3, f=8	(4,1)[f=8] , (3,1)[f=8] , (0,1)[f=8]

Step	Expand	Neighbours added to the queue	Priority queue after the step (best first)
6	(4,1) g=5, h=3, f=8	none (all neighbours blocked or already expanded: dead end)	(3,1)[f=8] , (0,1)[f=8]
7	(3,1) g=4, h=4, f=8	(3,2) g=5, h=3, f=8	(3,2)[f=8] , (0,1)[f=8]
8	(3,2) g=5, h=3, f=8	(2,2) g=6, h=4, f=10	(0,1)[f=8] , (2,2)[f=10]
9	(0,1) g=1, h=7, f=8	(0,2) g=2, h=6, f=8	(0,2)[f=8] , (2,2)[f=10]
10	(0,2) g=2, h=6, f=8	(1,2) g=3, h=5, f=8 (0,3) g=3, h=5, f=8	(1,2)[f=8] , (0,3)[f=8] , (2,2)[f=10]
11	(1,2) g=3, h=5, f=8	(2,2) g=4, h=4, f=8 note: (2,2) improved: g 6 → 4, old entry (f=10) discarded	(2,2)[f=8] , (0,3)[f=8]
12	(2,2) g=4, h=4, f=8	(2,3) g=5, h=3, f=8	(2,3)[f=8] , (0,3)[f=8]
13	(2,3) g=5, h=3, f=8	(2,4) g=6, h=2, f=8	(2,4)[f=8] , (0,3)[f=8]
14	(2,4) g=6, h=2, f=8	(1,4) g=7, h=3, f=10 (3,4) g=7, h=1, f=8	(3,4)[f=8] , (0,3)[f=8] , (1,4)[f=10]
15	(3,4) g=7, h=1, f=8	(4,4) g=8, h=0, f=8	(4,4)[f=8] , (0,3)[f=8] , (1,4)[f=10]
16	(4,4) g=8, h=0, f=8	Goal reached	–

Reading the trace

- From step 1 every node on the left and bottom side has f = 8, equal to the straight-line Manhattan lower bound, so the tie-break rules decide the order. A* first works down the left column and the bottom-left pocket (which ends in the dead end at (4,1)), and only returns to (0,1) at step 9.
- At step 8 the pocket reaches (2,2) with g = 6 (f = 10). At step 11 a cheaper route to (2,2) with g = 4 is found through (1,2), so the old entry is discarded and the new one expanded.
- The goal is dequeued at step 16 with g = 8. A* expanded 16 nodes in total.

2.3 Optimal path and cost

Result

Path: (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4)

Moves: Right, Right, Down, Down, Right, Right, Down, Down

Total cost = 8

This is optimal: the path cost equals h(S) = 8, which is a lower bound on any route from S to G, so no cheaper path exists.

	c0	c1	c2	c3	c4
r0	S	1	2		
r1		#	3	#	
r2		#	4	5	6
r3				#	7
r4			#		G·8

Blue cells show the optimal path with step numbers

2.4 BFS on the same grid (for comparison)

BFS expands nodes level by level using a FIFO queue. With unit step costs, the first time it reaches G is along a shortest path.

Depth	Nodes dequeued, in FIFO order
0	(0,0)
1	(1,0), (0,1)
2	(2,0), (0,2)
3	(3,0), (1,2), (0,3)
4	(4,0), (3,1), (2,2), (0,4)
5	(4,1), (3,2), (2,3), (1,4)
6	(2,4)
7	(3,4)
8	(4,4)

Algorithm	Queue	Nodes expanded	Path cost
BFS	FIFO	19	8
A* (Manhattan)	Priority (f = g + h)	16	8

Both return the same optimal cost of 8. A* expands fewer nodes because the heuristic keeps the search pointed towards the goal; the gap becomes larger on bigger grids.

Question 3: Autonomous Rescue Robot (Online Search)

3.a Effect of each constraint on decision-making

Constraint	Effect on the robot's decisions
Moves: up, down, left, right	The state space is the set of grid cells and each state has at most 4 successors. Every move can be undone, so the robot can retreat from a dead end and replan. A move cannot jump over a blocked cell.
Blocked cells	Hard constraint: the robot must never enter them. A path that looked valid on an incomplete map can become invalid when a blocked cell is discovered, so plans must be revisable.
Limited sensing (adjacent cells only)	The environment is partially observable. The robot keeps a knowledge map and can only plan on what it knows plus assumptions about unseen cells. Each step reveals new information, which creates a loop of sense, plan and act.
Move cost 1, risky zone +2	Costs are no longer uniform: entering a risky cell costs 3. BFS and DFS, which count steps, are no longer optimal, so a cost-aware search (UCS or A*) is required. On a grid every alternative route has the same parity of length, so a detour adds 0, 2, 4, … moves. One risky cell (+2) therefore costs the same as the shortest possible detour, and ties are common.
Minimise total cost	Defines the performance measure: the robot compares whole paths by total cost and not by number of moves or by closeness to the goal.
Safe navigation	Blocked cells are a hard constraint, and risky cells are a soft constraint expressed through the +2 penalty. Because the robot senses all four neighbours before each move, the cell it steps into is always one it has already observed, so it never walks into an obstacle blindly.
Dynamic knowledge update (online search)	The robot cannot compute a full plan offline. It must interleave planning and acting, update its map after every percept, and replan whenever new information changes the best route.

3.b Solution approach: online Uniform Cost Search with replanning

The robot runs a plan–sense–act loop. At every step it applies Uniform Cost Search (UCS) to its current knowledge map, executes only the first action of the plan, senses again, and replans. Unknown cells are assumed to be free (the free-space assumption), which keeps the plan optimistic and ensures the robot always has a route to try.

Steps

1. **Initialise.** The knowledge map contains only the grid size, S and G. Unknown cells are treated as free with cost 1.
2. **Sense.** Observe the four adjacent cells and record each as free, blocked or risky.
3. **Plan.** Run UCS from the current cell to G on the knowledge map. Blocked cells are removed, entering a risky cell costs 3 and any other cell costs 1. Equal-cost ties are broken by insertion order.
4. **Act.** Move along the first step of the plan and add its cost to the running total.
5. **Repeat** steps 2–4 until the robot stands on G. If UCS finds no route even under the optimistic assumption, G is unreachable and the robot reports failure.

```
ONLINE-UCS-AGENT(S, G):
  known <- empty map;  pos <- S;  total <- 0
  while pos != G:
    known <- known + SENSE(adjacent cells of pos)
    plan  <- UCS(pos, G, known)      # unknown = free, risky = 3
    if plan is empty: return FAILURE   # G unreachable
    next  <- plan[1]
    total <- total + cost(next)
    pos   <- next
  return total
```

Worked example

Sample building (assumed): blocked cells (1,1), (2,3), (3,0); risky cells (4,2), (3,3). The robot starts knowing none of these. The left grid shows the true map and the right grid shows the route the robot actually took (numbers are the order of visits).

True map

	c0	c1	c2	c3	c4
r0	S				
r1		#			
r2				#	
r3	#			R	
r4			R		G

Route taken by the robot

	c0	c1	c2	c3	c4
r0	S				
r1	1	#			
r2	2	3		#	
r3	#	4		R	
r4		5	R·6	7	G·8

= blocked, R = risky (cost 3), blue = visited cells with order of visit

#	At	Newly sensed	Plan changed?	Est. remaining cost	Move	Step cost	Total
1	(0,0)	nothing new	first plan	8	(0,0) → (1,0)	1	1
2	(1,0)	(1,1) blocked	No	7	(1,0) → (2,0)	1	2
3	(2,0)	(3,0) blocked	Yes	6	(2,0) → (2,1)	1	3
4	(2,1)	nothing new	No	5	(2,1) → (3,1)	1	4
5	(3,1)	nothing new	No	4	(3,1) → (4,1)	1	5
6	(4,1)	(4,2) risky	No	5	(4,1) → (4,2)	3	8
7	(4,2)	nothing new	No	2	(4,2) → (4,3)	1	9
8	(4,3)	(3,3) risky	No	1	(4,3) → (4,4)	1	10

What happened in the run

- At (1,0) the robot senses that (1,1) is blocked. The plan down the left column is unaffected.
- At (2,0) it senses that (3,0) is blocked. The old plan is invalid, so UCS replans through (2,1) and (3,1).
- At (4,1) it senses that (4,2) is risky. The estimate rises from 3 to 5. A detour through row 3 is also estimated at 5, because the robot cannot yet know that (3,3) is risky, so the tie is broken towards (4,2).
- The robot reaches G with a total cost of **10** (8 moves, one of which enters a risky cell at cost 3).

Cost compared with full knowledge

With the complete map known in advance, the best route is (0,0) → (0,1) → (0,2) → (1,2) → (1,3) → (1,4) → (2,4) → (3,4) → (4,4) at cost **8**. The online robot paid 10, a ratio of 1.25. The extra cost is the price of partial observability: the risky cell at (4,2) was not visible until the robot reached (4,1).

3.c AI concepts demonstrated

Intelligent agent (PEAS description)

PEAS	Rescue robot
Performance measure	Reach the survivor at G with minimum total path cost, never enter blocked cells, and avoid risky zones where the cost saving does not justify them
Environment	Grid of rooms with blocked cells and risky zones, partially known to the robot
Actuators	Move Up, Down, Left, Right
Sensors	Detect whether each of the four adjacent cells is free, blocked or risky; know own position and the goal position

Agent type: a goal-based and utility-based agent with internal state. The knowledge map is its internal state (model of the world). The goal is to reach G. The utility is the negative of total path cost, which lets it trade a longer route against a risky shortcut. The loop sense → update model → search → act is its agent function, mapping the percept history to an action.

Rationality

A rational agent selects, at each step, the action expected to maximise its performance measure given the percept history and its built-in knowledge. The robot meets this definition because it:

- uses every percept: each newly sensed cell is stored and used in the next plan;
- chooses actions by least total cost, using UCS with cost 1 per move and 3 for a risky cell;
- treats blocked cells as forbidden and risky cells as costly, which encodes safety in the performance measure;
- revises its plan when new information contradicts the old one.

Rationality is not omniscience. The run above costs 10 against a hindsight optimum of 8, but the robot could not have known about the risky cell at (4,2) in advance. Given what it knew at each step, every decision was the best available, so the behaviour is rational.

Environment type

Property	Classification	Reason
Observability	Partially observable	Only adjacent cells are sensed, so the full map is hidden
Agents	Single agent	No other agents act on the environment
Determinism	Deterministic	A move into an open cell always succeeds, and the same cell always has the same status
Episodic or sequential	Sequential	Each move changes the position and so affects all later costs
Static or dynamic	Static (assumed)	The map does not change while the robot plans. If rubble shifted, the same replanning loop would handle it
Discrete or continuous	Discrete	Finite cells and a finite set of moves
Known or unknown	Unknown map, known rules	The effect of each action is known, but the layout must be learned by exploring

3.d Why this approach is the most suitable

Alternative	Why it is not the best fit
BFS / DFS	Count steps and ignore the +2 risk penalty, so they can return a path that is short but costly. DFS may also wander far from the goal.
Greedy best-first	Follows the heuristic only and ignores cost already paid, so it can walk through risky zones for a small gain and is not optimal.
Offline UCS / A* on the full map	Optimal but needs the whole map before moving, which the robot does not have.
Online UCS with replanning (chosen)	Cost-aware and optimal with respect to what the robot currently knows, and revises its plan with each new percept.

- **Handles non-uniform costs.** UCS expands nodes in order of path cost g(n), so it correctly trades off a risky shortcut against a safe detour, which BFS cannot do.
- **Handles partial observability.** Re-running the search after each sensing step lets the robot use new information at once, which is the definition of an online search agent.
- **Safe by construction.** Blocked cells are removed from the search space, risky cells are penalised, and the cell the robot steps into is always one it has already sensed.
- **Terminates.** On a finite grid, each replanning is triggered by newly revealed information, so the robot cannot loop forever. If no path exists, UCS reports failure.
- **Cheap to run.** Grids are small, so re-running UCS at every step is inexpensive. If speed matters, replacing UCS with A* using the Manhattan heuristic keeps the same optimality guarantee, because the heuristic is admissible and consistent under costs of at least 1.